

面向游戏 QA 的多 Agent 编排平台

把一句自然语言任务交给一群专职 Agent，从用例生成一路跑到回归报告

- 多 Agent
- 编排
- 规格驱动
- 工程化

最近更新 2026-09-16

- 下载 PDF
- 扫读版 deck

专职 Agent 十余个	内置工具 数十个	管控 Hook 近二十道	主链路 四步
已建档能力规格 四十余项	工程变更记录 百余条		
指标已脱敏，均为区间或量级表述			

它是什么

一句话：**对游戏 QA 说一句自然语言任务，拿到一份可复现的测试结论。**

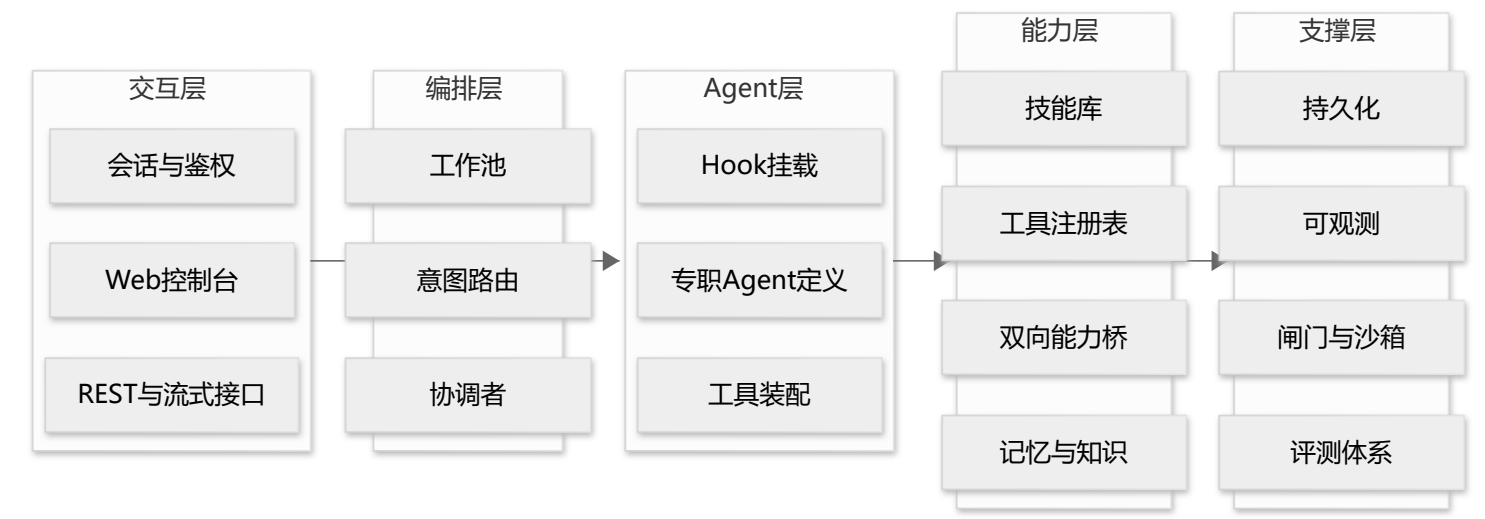
平台把这件事拆成四个步骤——规划、环境就绪、执行、收尾——每一步交给专职 Agent 完成；工具调用受管控，过程全留痕，结论可评

测。它的使用者是游戏测试与研发同学， 解决的问题是“一轮版本回归的链路太长，人被迫当调度器”。

问题背景

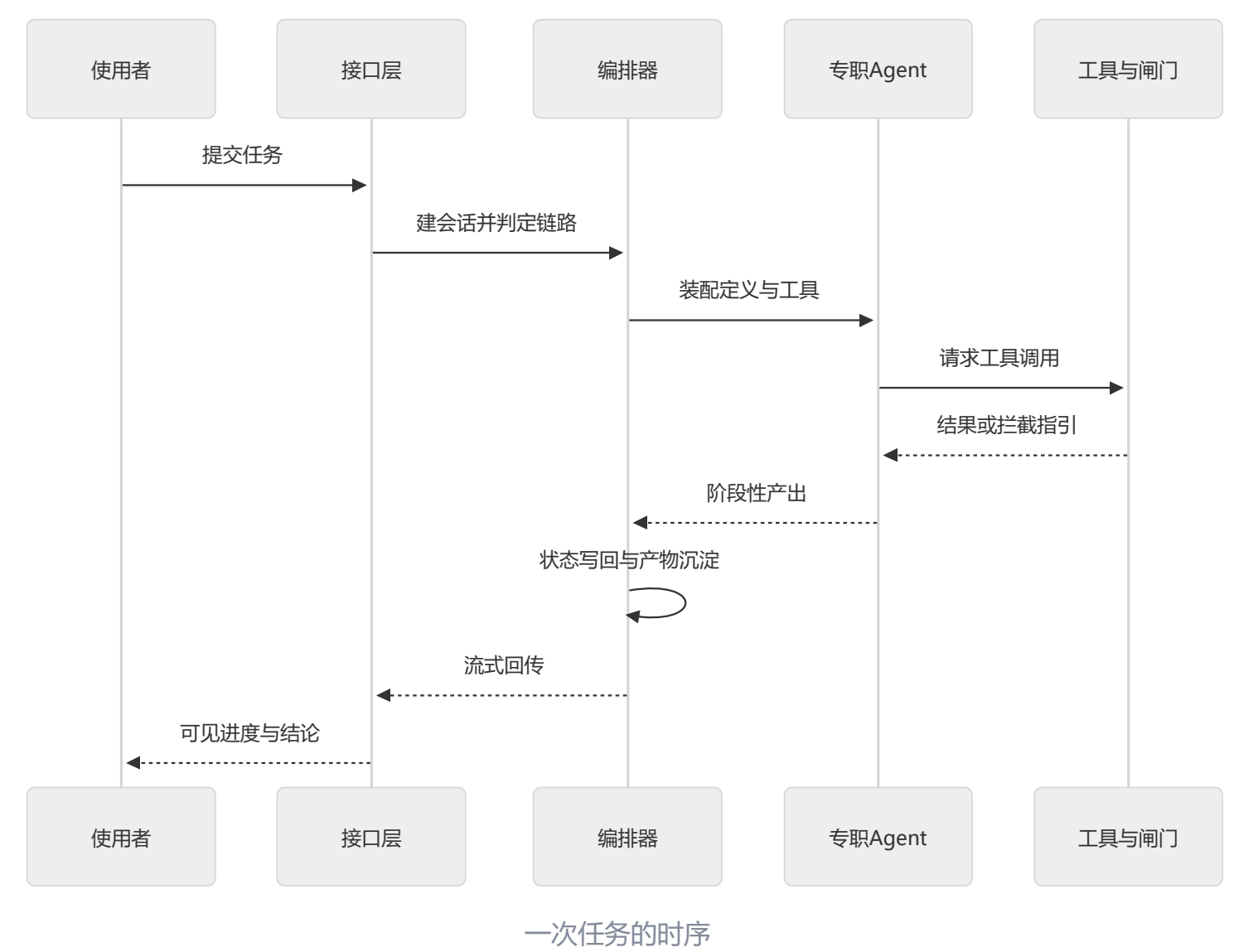
- **链路长**：起客户端、切版本分支、改配置、跑用例、收日志、写报告， 每一步都要人串起来，一轮回归的人力按天计
- **现场还原难**：游戏侧的失败信息散落在客户端日志、服务端日志与用例三处， 同一个现象每次都要重新拼一遍现场
- **经验留不下来**：判断依据留在人的记忆里，下一个版本没人知道上次为什么这么判

架构总览



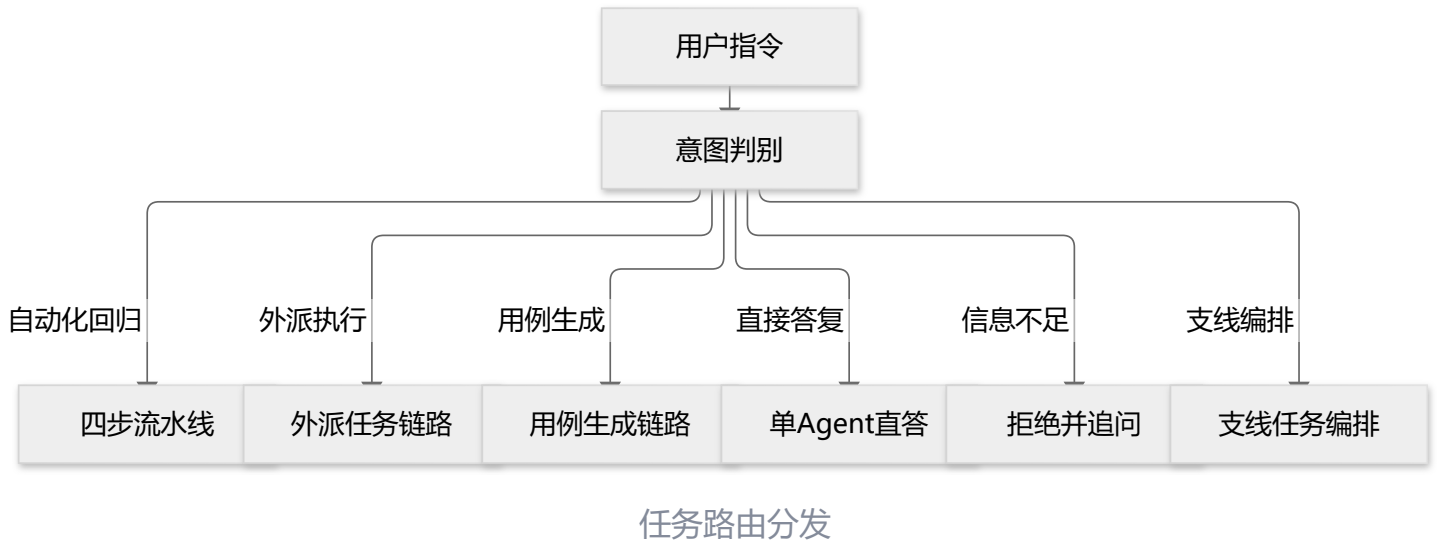
平台五层架构总览

一次任务从右侧进入：接口层建会话，编排层判定该走哪条链路（见下图）， Agent 层按定义装配工具与管控钩子，能力层提供工具、桥接、技能与记忆，支撑层负责留痕、审计与评测。**自上而下是调用关系，自下而上是反馈关系**—— 每一步的产物都会写回共享状态，成为下一步或下一轮的输入。



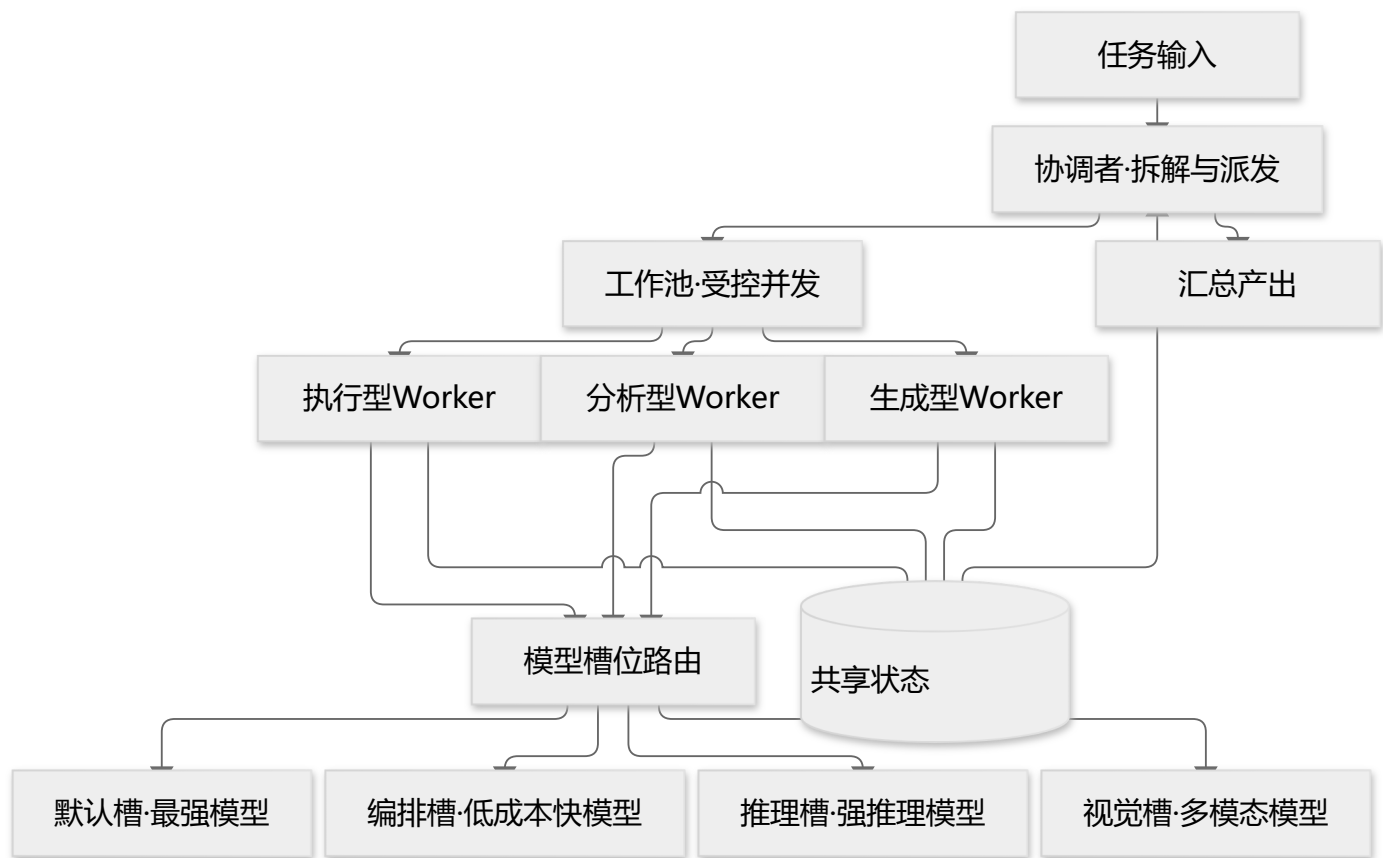
分层要点

编排层：先把任务分类，再决定怎么跑



- 指令先经**意图判别**分流：自动化回归、外派执行、用例生成、直接答复、拒绝并追问、支线编排
- 只有判为“信息不足”才拒绝，并在拒绝时说明缺什么——避免把可执行任务误判成不可执行
- 协调者负责拆解与派发，工作池负责受限并发：能并行的并行，但共享资源（设备、环境）串行
- 每一步的状态写回同一份会话状态，链路可中断、可恢复，不依赖单进程内存

Agent 层：一个任务由多个专职 Agent 接力



Agent 编排与模型槽位

- 每个 Agent 有独立定义：角色说明、可用工具集、权限级别、上下文注入策略
- 模型按**槽位**路由：编排用低成本快模型、推理用强推理模型、图像理解用多模态模型——把“贵模型”留给真正需要的判断，而不是所有环节都用同一个
- Agent 之间不直接对话，通过共享状态与产物交接，任何一步失败都能定位到具体环节
- 新增 Agent 是加一个定义文件并注册，不改动编排层代码

能力层：双向桥接，能力按需到岗

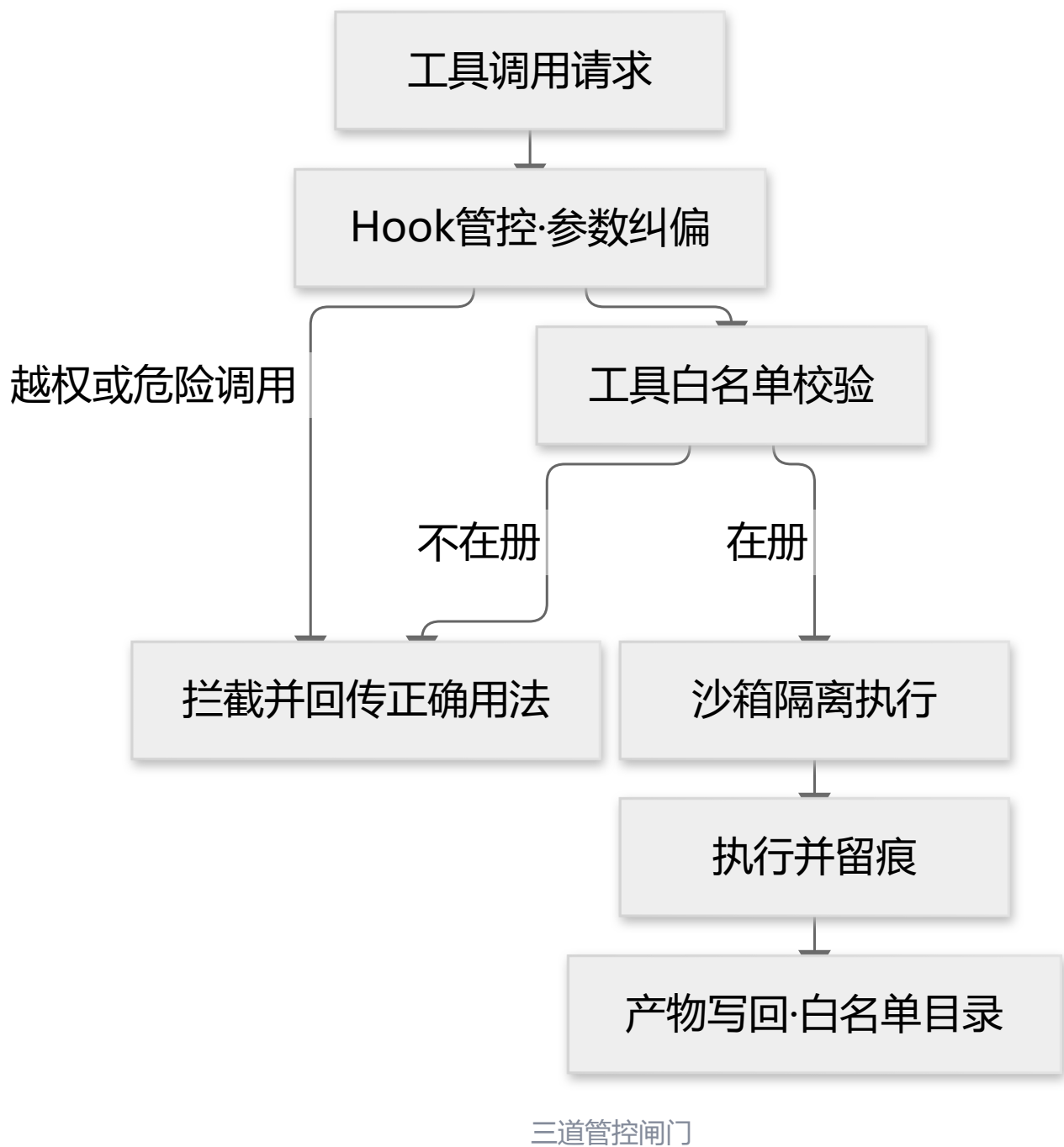


双向能力桥接

- **方向一：** 外部能力按需拉起，就绪探活通过后才注入工具——避免“工具在但服务没起来”的假可用
- **方向二：** 平台自身能力反向暴露为标准协议服务，供其它系统调用
- 技能库把“某类任务怎么做”沉淀成可复用资产，与代码解耦，改流程不用发版

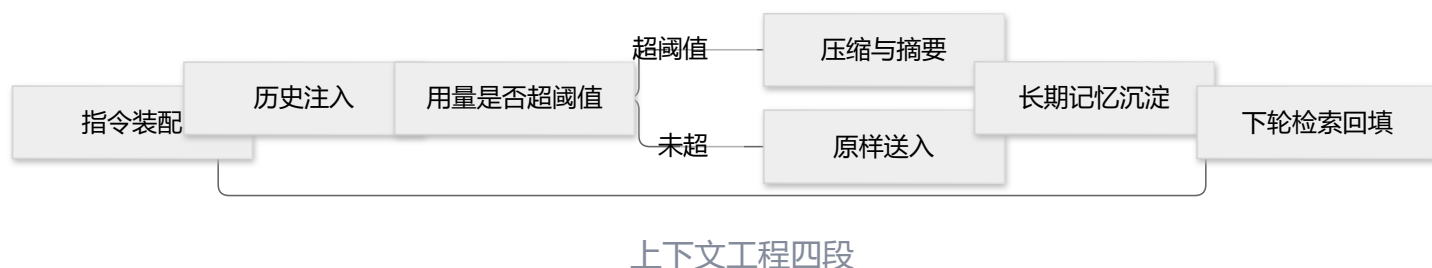
- 记忆与知识层负责长期沉淀：本轮结论被压缩成条目，下一轮按需检索回填

管控与安全：三道闸门，而不是一段提示词



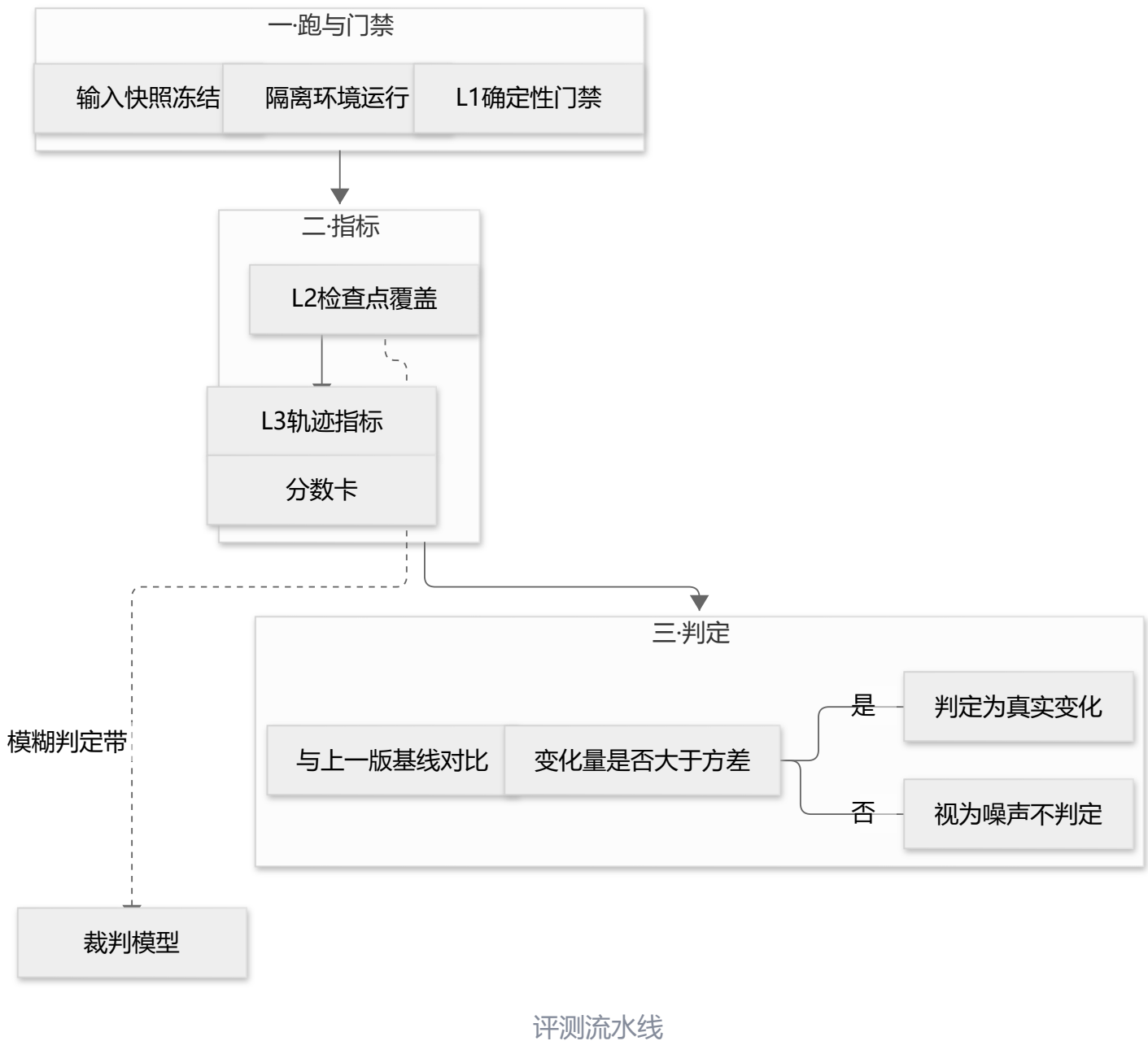
- **钩子层**：在调用现场纠偏与拦截——参数写错就地纠正，越权与高危调用直接拦下并回传正确用法
- **工具白名单**：每个 Agent 只能调用被显式授权的工具，越界调用被硬拦截，不靠模型自律
- **沙箱与审批**：写操作限制在授权目录内；不可逆动作（改文件、执行代码）需要人工审批放行
- 这套约束是**代码级**的，不占上下文 token，也不依赖模型记住规则

上下文工程：让长任务跑得下去



- 四段式：指令装配 → 历史注入 → 阈值压缩 → 长期沉淀并回填
- 用量超过阈值时才压缩，未超阈值保持原文——避免“无脑摘要”丢细节
- 关键约束（不可越界的红线、必须遵守的输出格式）不参与压缩，始终原样保留
- 长时任务可连续运行数小时级不中断，状态与会话解耦

评测与可观测：结论要可复现，改动要可判定



- 评测与生产解耦：冻结输入快照，在隔离环境重跑，不触发任何生产副作用
- 三层指标：确定性门禁（格式与事实）、检查点覆盖（人工标杆比对）、轨迹指标（路径是否走对）
- 模糊判定带交给裁判模型，裁判本身要校准——否则打分漂移会被误读成系统变好或变坏

- **方差是一等指标：**变化量小于基线波动一律判为噪声，不发布“提升”结论

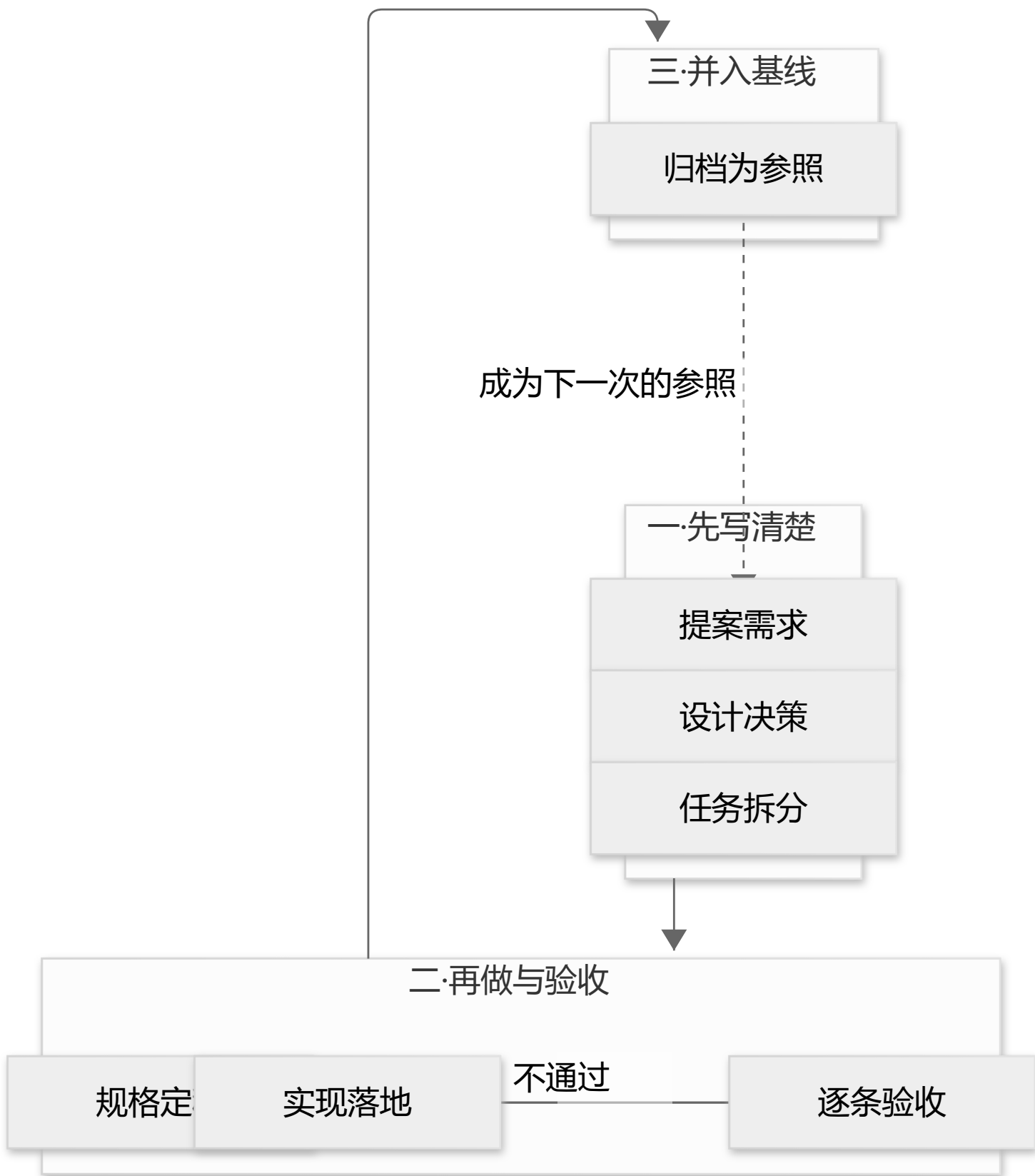


- 三类留痕：会话事件流、工具调用日志、状态变更记录
- 汇聚成跨层追踪与关键产物快照，排障与评测取材共用同一份数据

取舍与难点

取舍一：一群专职 Agent，而不是一个全能 Agent。 一个 Agent 也能做完整链路，但工具集会膨胀到几十个，模型每次都要在无关工具里选，错误率随工具数上升；而专职化让每个 Agent 的工具集收敛到个位数。代价是编排成本：多了路由与交接，链路更长，调试时要先定位是哪一环出的问题。→ 详见 [为什么拆成多个专职 Agent，而不是把它们合成一个](#)

取舍二：把工程流程套在 Agent 系统上，短期更慢。 每个功能先写提案与规格、拆任务、定验收，再动手实现——写代码的时间被流程吃掉一部分。换来的是：半年后回看任意一处设计，都能找到“当时为什么这么定”；多人并行时冲突在规格层就暴露，而不是在代码里互相覆盖。代价是它要求纪律：规格不更新，流程就退化成摆设。→ 详见 [用规格驱动的流程，管住一个持续演化的多 Agent 系统](#)



工程流程

取舍三：管控写进代码，而不是写进提示词。 提示词里写“不要执行危险命令”，模型大概率会遵守——但也只是大概率，且每次调用都要为这段约束付 token。改成在调用现场拦截后，约束变成确定性的，还能回传“正确的用法”引导模型自我纠正。代价是它与运行时强耦合：换掉编排框架，这层管控要重写。→ 详见 [三道闸门：怎么让 Agent 不越权、不幻觉、不闯祸](#)

我的角色与边界

- **我负责：**Agent 体系与编排机制、工具与外部能力桥接、管控闸门与沙箱约束、上下文与记忆机制、可观测与评测体系、以及这套工程流程本身的落地
- **我参与：**用例内容与业务知识库的构建，与使用方共同定义“什么算通过”
- **不是我做的：**被测业务的底层实现、基础设施与集群运维、平台之外的数据看板
- 这套系统是团队协作的产物：我负责的是**平台侧机制**，业务侧内容由使用者与领域同学提供

结果与指标

以下指标均为**量级与区间**表述，非精确内部数值。

维度	结果
专职 Agent	十余个，覆盖规划、环境、执行、收尾、分析、生成等职责
内置工具	数十个，按 Agent 职责分组授权
主链路	四步，任一步失败不阻塞收尾步骤执行
回归执行耗时	单轮回归从人天级降到小时级（对象是“跑完一轮回归”的总耗时）
沉淀用例量	数百条，可被后续任务检索复用
工程规模	四十余项能力规格，百余条变更记录，全部可追溯到提案与验收
长时任务	可连续运行数小时级，中断后可恢复

指标口径为“从无到有”的对比：即从纯人工串联，到交由平台自动执行同一条链路。

深挖入口

- **扫读版**：12 页概览 deck（本页顶部入口），3 分钟看完问题、方案与结果
- **离线版**：本页 PDF（本页顶部入口），适合邮件附件与打印
- **深挖文章**（六篇，每篇一个取舍）：
 1. [为什么拆成多个专职 Agent，而不是把它们合成一个](#)

2. [用规格驱动的流程，管住一个持续演化的多 Agent 系统](#)
3. [双向桥接的取舍：能力该常驻，还是按需拉起](#)
4. [三道闸门：怎么让 Agent 不越权、不幻觉、不闯祸](#)
5. [上下文工程：什么时候该压缩，什么时候绝不能压](#)
6. [让模型给人打分之前，先校准这把尺子](#)

想聊机制细节或看某一段的设计取舍，欢迎邮件联系（见[关于](#)）。

786939553@qq.com · [代码托管](#)

© 2026 文棚嶺